

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_7

7. Singleton Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Singleton pattern.

GoF Definition

It ensures that a class has only one instance and provides a global point of access to it.

Concept

Let's assume you have a class called `A` and you need to create an object from it. Normally what do you do? You guess it right; you simply use the following line of code: `A obA = new A();`

But let's look at it closely. If you keep using the `new` keyword ten more times, you'll have ten more objects. Right? But in a real-world scenario, unnecessary object creation is a big concern (particularly when constructor calls are truly expensive), so you need to restrict this. In a situation like this, the Singleton pattern comes into the picture. It restricts the use of `new` like this and ensures that you do not have more than one instance of the class.

In short, this pattern says that a particular class should have only one instance. You can create an instance if it is not available; otherwise, you should use an existing instance to serve your need. Following this approach, you can avoid creating multiple objects.

Real-Life Example

Let's assume that you have a sports team, and your team is participating in a tournament. Your team needs to play against multiple opponents

throughout the tournament. At the beginning of each match, as per the rules of the game, the captains of the two sides must go for a coin toss. So, if your team does not have a captain, you need to elect someone as a captain first. Prior to each game and each coin toss, you may not repeat the process of electing a captain if you have already nominated a person as a captain of your team for this tournament. Notice that in this example, selecting a captain is like creating a new object. Having that captain for the rest of the season is like reusing the same object that was created at the beginning.

Computer World Example

In some software systems, you may decide to maintain only one file system so that you can use it for the centralized management of resources. This approach can help you to implement caching mechanisms effectively. This pattern can also be used to maintain a thread pool in a multithreading environment. Here is another example for you:

- Once you learn about eager initialization in this chapter, you'll recognize that the `java.lang.Runtime` class follows this pattern.

Implementation

A Singleton pattern can be implemented in many different ways. Each approach has its pros and cons. We'll examine some of them.

The first demonstration takes a simple approach. Let's go through the notable characteristics of this approach:

- The `Captain` class is implemented as a singleton where its constructor is `private`. So, you cannot instantiate this class outside. For example, inside the client code, you cannot write something like the following:

```
Captain captain = new Captain(); // error
```

- This construct will help you refer to the only instance that can exist in the system.
- The `private` constructor also ensures that the `Captain` class cannot be extended. So, subclasses cannot misuse the concept. Thus, the following line of code also produces a compile-time error:

```
// We cannot extend Captain class.  
// The constructor is private in this case.
```

```
class B extends Captain{} // error
```

- `getCaptain()` is responsible for the `Captain` instance creation. This method is synchronized, which means that multiple threads cannot involve in the instantiation process at the same time. It enables you to force each thread to wait for its turn to access this method. So, thread-safety is ensured.
- Although it was not strictly necessary for this implementation, I marked the `Captain` class as final to guard a specific scenario. I discuss the reason in the Q&A session.

Note Synchronization is a costly operation and once the instance is created, it is an additional overhead. I discuss some alternative methods in the upcoming sections. Each has its pros and cons.

Class Diagram

Figure 7-1 shows the class diagram for the illustration of the Singleton pattern.

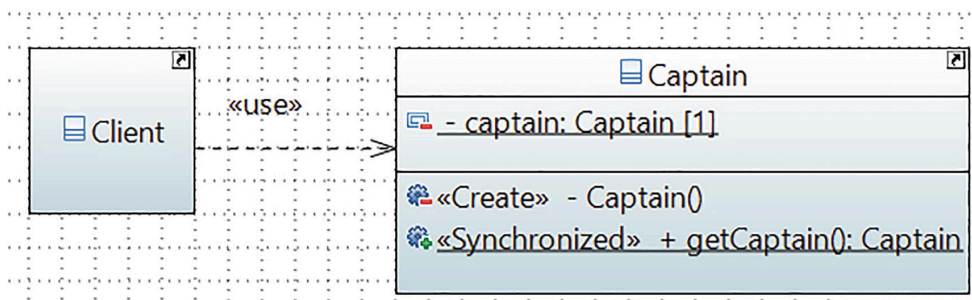


Figure 7-1 Class diagram

Package Explorer View

Figure 7-2 shows the high-level structure of the program.

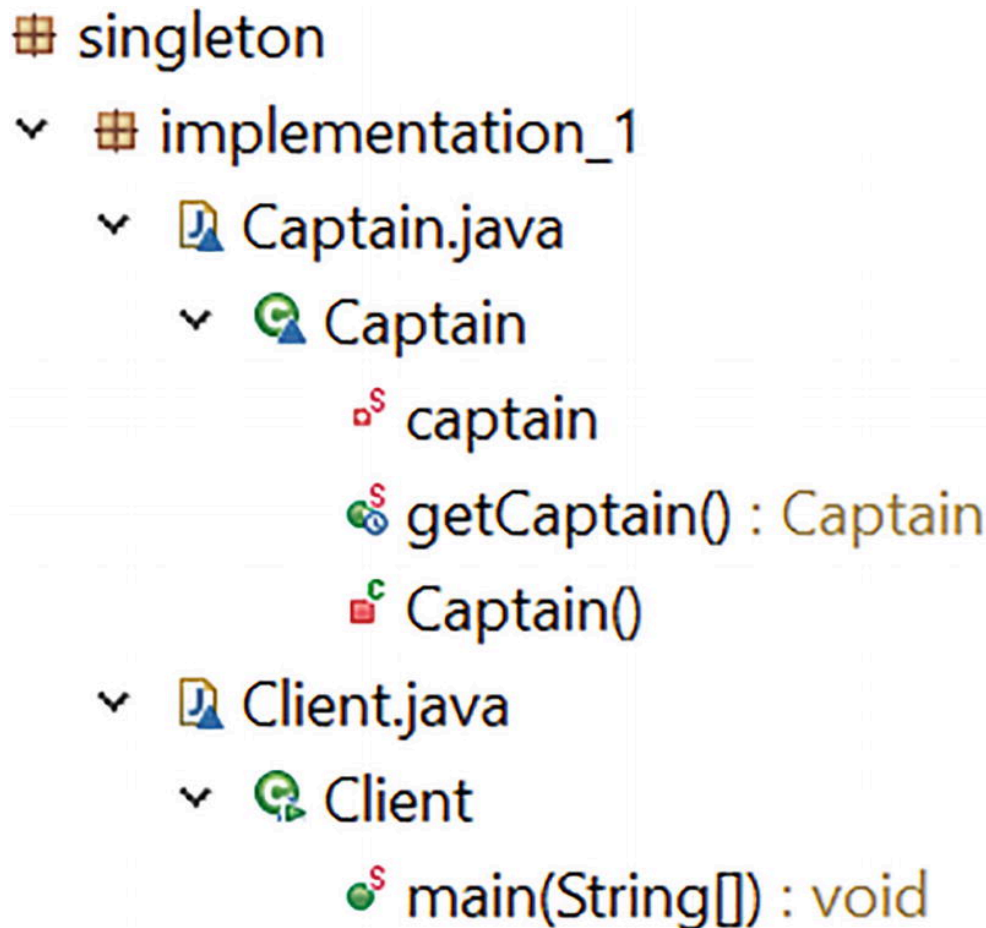


Figure 7-2 Package Explorer view

Demonstration 1

Now go through the following implementation. There are supportive comments for your easy understanding. All parts of the program are stored inside the package named `jdp3e.singleton.implementation_1`.

```
// Captain.java
// This class is declared as 'final'.
// So, it cannot have a subclass.
final class Captain {
    private static Captain captain;
    // Making the constructor private
    // to prevent the use of "new"
    private Captain() { }
    public static synchronized Captain getCaptain() {
        // Lazy initialization
        if (captain == null) {
            captain = new Captain();
            System.out.println("\tA new captain is elected for your team.")
        }
        else {
            System.out.println("\tYou already have a captain for your team")
            System.out.println("\tSend him for the toss.");
        }
    }
}
```

```
        return captain;
    }
}
// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Singleton Pattern Demo***\n");
        System.out.println("Trying to make a captain for your team.");

        Captain captain1 = Captain.getCaptain();
        System.out.println("Trying to make another captain for your team.");
        Captain captain2 = Captain.getCaptain();
        if (captain1 == captain2){
            System.out.println("Both captain1 and captain2 are the same.");
        }
    }
}
```

Output

When you execute this program, you'll see the following output:

```
***Singleton Pattern Demo***
Trying to make a captain for your team.
    A new captain is elected for your team.
Trying to make another captain for your team:
    You already have a captain for your team.
    Send him for the toss.
Both captain1 and captain2 are the same.
```

Analysis

In demonstration 1, you saw a common way to implement the Singleton pattern. If you analyze further, you will see that the overall process can be boiled down into the following steps:

- Make the constructor `private`.
- Use a `public static` getter method that returns the static instance of the Singleton class.

Q&A Session

7.1 Why do you use the `synchronized` keyword? Looks like I can write the same program without using it.

This approach can work in a single-threaded environment. But consider a multithreaded environment where two (or more) threads may try to evaluate the following code at the same time:

```
if (captain == null)
```

If they see that the instance has not been created yet, each of them will try to create a new instance. As a result, you may end up with multiple instances of the class.

7.2 Why do you use the term lazy initialization in the code?

It's a technique that you use to delay the object creation process. The basic idea is that you should create the object only when it is truly required. This method is useful when object creation is a costly operation. So, using lazy initialization, you delay the object creation process to avoid expensive processes to create an object.

7.3 In short, if I need to create synchronized code, I can use the `synchronized` keyword in Java. Is my understanding correct?

Yes, JVM ensures this. Internally it uses locks on a class or an object to ensure that only one thread is accessing the data. Depending on an implementation, you may see the use of synchronized methods as well as synchronized statements.

7.4 Why are multiple object creations a big concern?

Here are some important points that you can remember:

- Object creations in the real world can be costly if you are working with resource-intensive objects.
- In some applications, you may need to pass a common object to multiple places.

7.5 Why are you making the class final? You have a private constructor that itself could prevent the inheritance. Is my understanding correct?

Subclassing can be prevented in various ways. Yes, in this example, since the constructor is already marked with the `private` keyword, it is not

needed. But if you make the `Captain` class final, as shown in the example, this approach can be considered a better practice. This approach can be effective when you consider a nested class.

For example, let's modify the private constructor body to examine the number of instances (of the `Captain` class) created. Let's further assume that in the above example, you have a non-static nested class (called an inner class in Java) like the following. (All changes are shown in bold. I have also placed this code in a separate package named `jdp3e.singleton.implementation_2`. It can help you test the code directly once you download it from the Apress website).

```
package jdp3e.singleton.implementation_2;

class Captain {
    private static Captain captain;
    static int numberOfInstance = 0;
    // Making the constructor private
    // to prevent the use of "new"
    private Captain() {
        numberOfInstance++;
        System.out.println("Number of instances at this
                           moment=" + numberOfInstance);
    }
    public static synchronized Captain getCaptain() {
        // Lazy initialization
        if (captain == null) {
            captain = new Captain();
            System.out.println("\tA new captain is elected for your team.");
        } else {
            System.out.println("\tYou already have a
                               captain for your team.");
            System.out.println("\tSend him for the toss.");
        }
        return captain;
    }
    // A non-static nested class (inner class)
    public class CaptainDerived extends Captain {
        // Some code
    }
}
```

Now add an additional line of code (shown in bold) inside the `main()` method:

```
import jdp3e.singleton.implementation_2.Captain;
class Client {
    public static void main(String[] args) {
        System.out.println("***Singleton Pattern Demo***\n");
        System.out.println("Trying to make a captain for your team.");
    }
}
```

```
// Constructor is private. We cannot use "new"
// here.
// Captain captain = new Captain(); // error
Captain captain1 = Captain.getCaptain();
System.out.println("Trying to make another captain for your team");
Captain captain2 = Captain.getCaptain();
if (captain1 == captain2){
    System.out.println("Both captain1 and captain2 are the same");
}

Captain.CaptainDerived derived1=captain1.new
    CaptainDerived();

Captain.CaptainDerived derived2=captain1.new
    CaptainDerived();

}
}
```

When you execute the program, you can see the following output:

```
***Singleton Pattern Demo***
Trying to make a captain for your team.
Number of instances at this moment=1
    A new captain is elected for your team.
Trying to make another captain for your team:
    You already have a captain for your team.
    Send him for the toss.
Both captain1 and captain2 are the same.
Number of instances at this moment=2
Number of instances at this moment=3
```

Have you noticed that the total number of instances is increasing? So, although in my original demonstration I could exclude the use of the `final` keyword, I kept it to guard against this type of situation, which may arise due to a modification of the original implementation.

7.6 When should I use the Singleton pattern?

Here are some common use cases where you'll find this pattern useful:

- To work with a centralized system (for example, consider a database).
- To maintain a common log file.
- To maintain a thread pool in a multithreaded environment.
- To implement a caching mechanism or device drivers and so forth.

Alternative Implementations

There are many approaches to implementing a Singleton design pattern. Each has its pros and cons. You have seen an example. Let's discuss some alternative approaches.

Demonstration 2: Eager Initialization

In eager initialization, an object of the singleton class is always instantiated at the beginning. The following code snippet shows such an example:

```
class Captain {
    // Early initialization
    private static final Captain CAPTAIN_INSTANCE =
                                                new Captain();

    // Making the constructor private
    // to prevent the use of "new"
    private Captain() {
        System.out.println("\tNew captain is elected for your team.");
    }
    // Global point of access.
    public static Captain getCaptain(){
        System.out.println("\tYou already have a captain for your team.");
        System.out.println("\tSend him for the toss.");
        return CAPTAIN_INSTANCE;
    }
}
```

Analysis

This approach has the following pros and cons.

Pros:

- This approach is straightforward and cleaner.
- It is opposite to lazy initialization but it is thread-safe.
- There is a small lag time when the application is already in execution mode because all kinds of stuff is already loaded in memory.

Cons:

- The application will take longer to start (compared to lazy initialization) because everything needs to be loaded first. To examine this, let's add a dummy method (shown in bold) in the singleton class. Notice that in the main method, I invoke only this dummy method. Now examine the output.

```

class Captain {
    // Early initialization
    private static final Captain CAPTAIN_INSTANCE =
                                                new Captain();

    // Making the constructor private
    // to prevent the use of "new"
    private Captain() {
        System.out.println("\tA new captain is elected
                            for your team.");
    }

    // Global point of access.
    public static Captain getCaptain(){
        System.out.println("\tYou already have a captain for your team.");
        System.out.println("\tSend him for the toss.");
        return CAPTAIN_INSTANCE;
    }

    public static void dummyMethod(){
        System.out.println("It is a dummy method");
    }
}

// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Singleton Pattern Demo
                            using Eager Initialization***\n");
        Captain.dummyMethod();
        // Remaining code is omitted
    }
}

```

Now run this program segment and notice the output:

```

***Singleton Pattern Demo using Eager Initialization***
    A new captain is elected for your team.
It is a dummy method

```

Note that “New captain is elected for your team” has appeared in this output, although you may have no intention of dealing with it.

Demonstration 3: Bill Pugh’s Solution

In the earlier days (prior to Java 5), there were many issues when dealing with singleton classes. To overcome those situations, Bill Pugh came up with a different approach using a static nested helper class:

```

class Captain {
    private Captain() {
        System.out.println("\tNew captain is elected for your team.");
    }
    // The helper class
    private static class SingletonHelper {
        // This nested class is referenced after
        // the getCaptain() method is called.
        private static final Captain CAPTAIN_INSTANCE =
            new Captain();
    }
    public static Captain getCaptain() {
        return SingletonHelper.CAPTAIN_INSTANCE;
    }
}

```

Analysis

This method does not use the synchronization technique and eager initialization. Notice that the `SingletonHelper` class comes into consideration only when someone invokes the `getCaptain()` method. And this approach will not create any unwanted output if you call any `dummyMethod()` inside the `main()` method, as shown in the previous case.

Note In all these different approaches of implementing a Singleton Pattern, the `main()` method is essentially the same. So, for brevity, I have not added this segment repeatedly in these discussions. But it is in the full code on the GitHub/Apress website. If interested, you can download the full implementations from there.

Demonstration 4: Enum Singleton

Joshua Bloch is an American software engineer and famous technical author. He shows us that we can use `enum` type(s) to implement a Singleton design pattern. I encourage you to watch his talk at www.youtube.com/watch?v=pi_I7oD_uGI#t=28m50s (particularly, listen to the talk from 29:00-32:00). He explains why an apparently good-looking singleton class can be broken by an intelligent hacker if the class has a non-transient field.

So, what is the suggestion? Here is a sample implementation for you:

```

// ENUM Singleton
enum Captain{

```

```
    INSTANCE;  
    public synchronized void getCaptain()    {  
        System.out.println("\tYou already have a captain for your team.");  
        System.out.println("\tSend him for the toss.");  
    }  
}
```

The previous code can be used effectively in the following client code:

```
class Client {  
    public static void main(String[] args) {  
        System.out.println("***Singleton Pattern Demo using enum type**");  
        Captain.INSTANCE.getCaptain();  
    }  
}
```

When you run this program, you get the following output:

```
***Singleton Pattern Demo using enum type**  
    You already have a captain for your team.  
    Send him for the toss.
```

Analysis

An `enum`-based Singleton class is easy to write compared to other traditional approaches. But at the same time, you need to keep in mind the restrictions that are associated with an `enum` type. Some of these restrictions can be tricky depending on the implementation.

The further details of each of these techniques and other approaches are beyond the scope of the book. The reason is simple: the book is very fat and you still have a long path to cover. At the same time, I also believe that you have got the idea behind the Singleton design pattern. I encourage you to read the online article www.journaldev.com/1377/java-singleton-design-pattern-best-practices-examples. Here you can learn more about various implementations of this pattern in brief.

Q&A Session Continued

7.7 I have one last question for you. I have some concerns about the example of eager initialization. Following the definition, it appears to me that it is also not exactly “eager.” I understand that this class will be loaded by the JVM only when it is referenced by some code during

the execution of the application. That way this is also lazy initialization. Is this understanding correct?

Yes, to some extent your observation is correct. There is a debate on this point. In short, it is *eager* compared to the previous approaches. You have seen that when you called the `dummyMethod()` only, still you instantiated the singleton, although you did not need it. So, in a context like this, it is *eager*, but it is *lazy* in a sense that the singleton instantiation will not occur until the class is initialized. So, the degree of laziness is the key concern here.

Lastly, I must say that “performance vs. laziness” is always a concern with this pattern, and some developers always question those areas. But the truth is that this pattern can be found in many applications. Let’s finish the chapter with an Erich Gamma quote from an interview in 2009

(<https://www.informit.com/articles/article.aspx?p=1404056>):

“When discussing which patterns to drop, we found that we still love them all. (Not really—I'm in favor of dropping Singleton. Its use is almost always a design smell.)”

Erich Gamma

Previous chapter

< [6. Builder Pattern](#)

Next chapter

[8. Proxy Pattern](#) >